Deep Learning in Computer Vision· Lecture 7 of 9

# Semantic Segmentation

Classifying every pixel: FCN, SegNet, U-Net, losses and metrics

# Where We Are, and Roadmap

- **Weeks 1-2**: a CNN maps a whole **image → one label** (classification). Today we need **where**, not only what: **one label for every pixel**.
    - **Part A, the task**: what segmentation outputs and why a plain CNN is too expensive.
    - **Part B, FCN and upsampling**: turning a classifier into a pixel-wise predictor and growing feature maps back to full size.
    - **Part C, SegNet and U-Net**: encoder-decoder networks with different ways of recovering detail.
    - **Part D, learning and measuring**: losses (cross-entropy, Dice) and metrics (pixel accuracy, IoU, mIoU).
- **Goal**: explain an encoder-decoder, upsampling, skip connections, and compute IoU / Dice by hand.

# Glossary: Terms Used in This Lecture

| Term | Meaning |
|---|---|
| Pixel | One picture element; an $H \times W$ image has $H \times W$ pixels |
| Class / label | Category assigned to a pixel (road, car, background …); $K$ = number of classes |
| Ground truth (mask) | The correct label map drawn by human annotators |
| Feature map | Output of a conv layer: a grid of values showing where a pattern was found |
| Downsampling / upsampling | Reducing / increasing the height and width of a feature map |
| Encoder / decoder | The shrinking half / the growing half of a segmentation network |
| Skip connection | A link that passes features from an earlier layer directly to a later one |
| Semantic vs. instance | Same class = same label / each object separate |
| argmax | Picks the index of the largest value, e.g. the class with the highest score |

# The Family of Recognition Tasks

| Task | Output | Example answer | Typical metric |
|---|---|---|---|
| **Classification** | One label per image | "street scene" | Accuracy |
| **Object detection** | Class + bounding box per object | 3 cars, 2 people, each in a box | mAP (Lecture 8) |
| **Semantic segmentation** | Class for every pixel | all car pixels = "car" | mIoU, Dice |
| **Instance segmentation** | Class + object ID per pixel | "car #1" and "car #2" separated | mask AP |
| **Panoptic segmentation** | Semantic + instance together | "stuff" (road, sky) and "things" (cars) | PQ (Panoptic Quality) |

- **Note: Things** are countable objects (cars, people); **stuff** is regions (sky, road, grass).

- Today: **semantic segmentation**.
  - Object Detection is Lecture 8; instance masks appear at the end of Lecture 8.

# What Is Semantic Segmentation?

- **Input**: image of size **H × W × 3**.
- **Output**: label map of size **H × W**; entry $(i, j)$ is the class of pixel $(i, j)$.
- The network produces $\boldsymbol{K}$ **channels** (one score map per class);
  - each pixel takes the class with the highest score:
    - $\hat{y}(i,j) = \boldsymbol{argmax_k}\, \boldsymbol{p_k}(i,j)$.
- Same class → same label: two cars are **not** distinguished

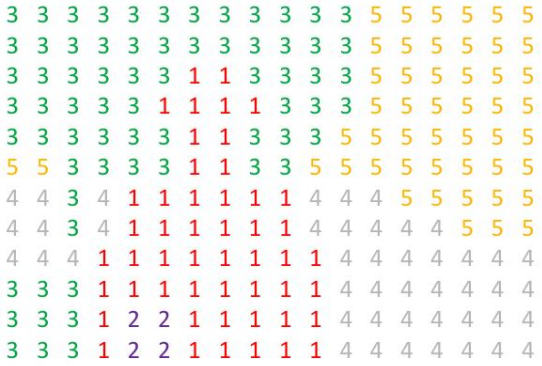
| Symbol / term | What it means |
|---|---|
| $H, W$ | image height and width in pixels |
| $K$ | number of classes (incl. background) |
| $p_k(i,j)$ | predicted probability that pixel $(i,j)$ has class $k$ |
| $\hat{y}(i,j)$ | final predicted class of that pixel (hat = prediction) |



Input

segmented

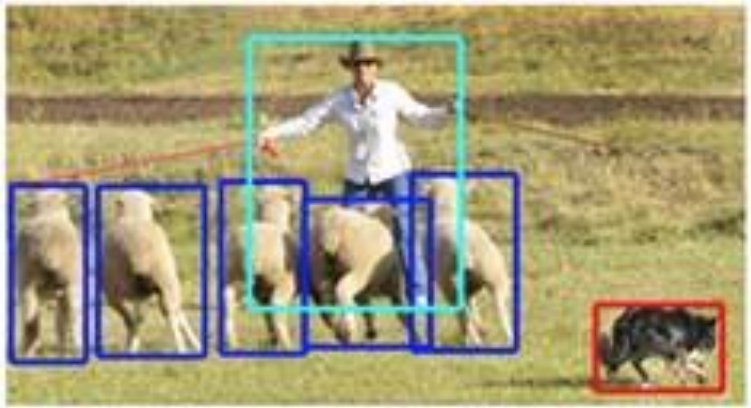
1: Person
2: Purse
3: Plants/Grass
4: Sidewalk
5: Building/Structures




Semantic Labels

# How Is It Different From Object Detection?

- **Detection**: a **bounding box** + label for each object; objects are **counted**, but the box also contains background pixels.
    - Detection answers "what and roughly where"; segmentation answers "which pixels belong to what".
- **Segmentation**: the **exact shape** of each region, pixel by pixel; no boxes, and no object count (semantic version).
- Choose by need: **counting people → detection**; **measuring tumor area → segmentation**.
    - Segmentation gives shape; detection gives location and identity



detection



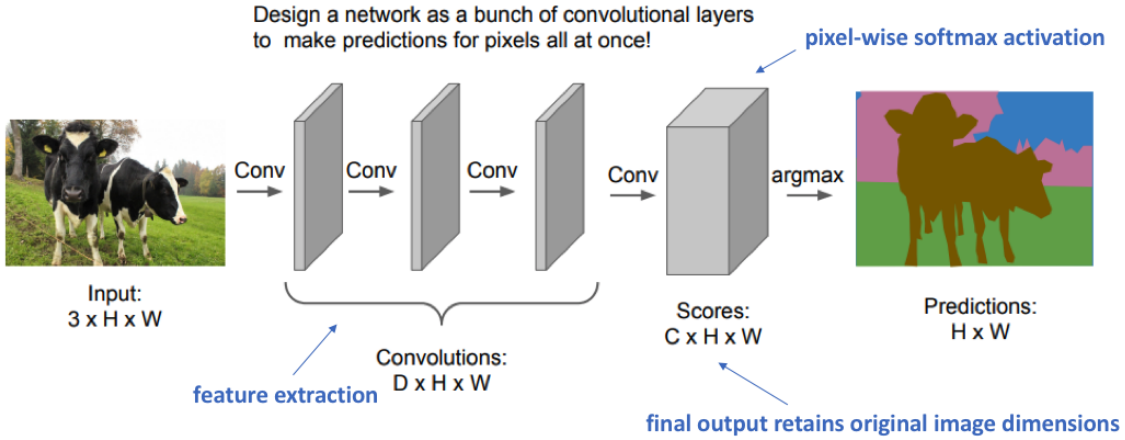
segmentation

# Where Is Segmentation Used?

| Field | What is segmented | Why pixel accuracy matters |
|---|---|---|
| Autonomous driving | road, lane, sidewalk, pedestrian | free-space planning needs exact road boundary |
| Medical imaging | organs, tumors, cells | tumor volume/area is computed from the mask |
| Remote sensing | land cover, buildings, crops | area statistics per class |
| Robotics, AR | objects, background | grasp points; background replacement |
| Industry | defect regions | defect size decides pass / fail |

# Why Not Just a Plain CNN at Full Resolution (No Downsampling) ?

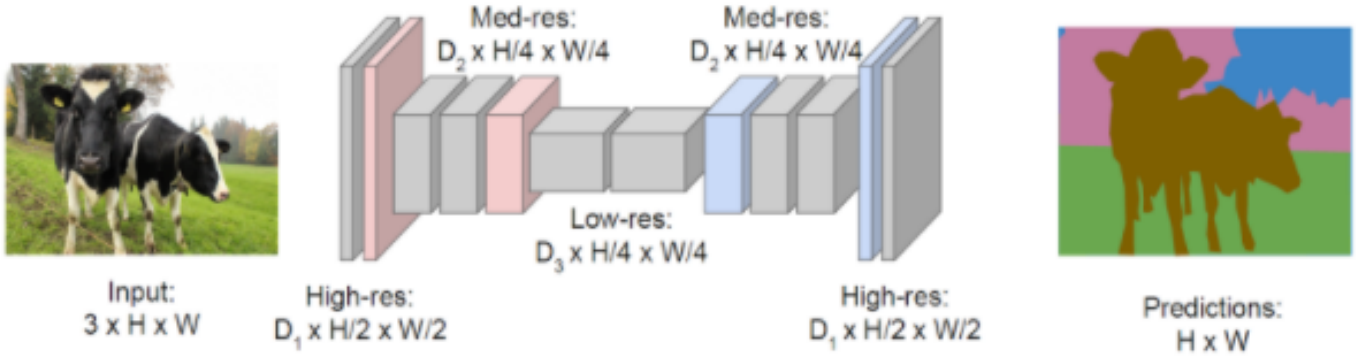
- **Naive design**: stack same-size convs so the output is H × W.
- **Memory**: **one** 512×512×64 **feature map** = **16.8M numbers** × 4 bytes = **≈ 67 MB** per layer, per image, plus gradients.
  - Too expensive for high-resolution (e.g. medical) images
- **Small context**: ten 3×3 convs (stride 1) see only 1 + 2×10 = **21×21** pixels; too little to tell a tumor from a shadow.
- **Solution:** shrink first (cheap, large receptive field), then grow back: **encoder-decoder**.




**Downside:** Preserving image dimensions throughout entire network will be computationally expensive.

# Encoder-Decoder: Downsample, Then Upsample

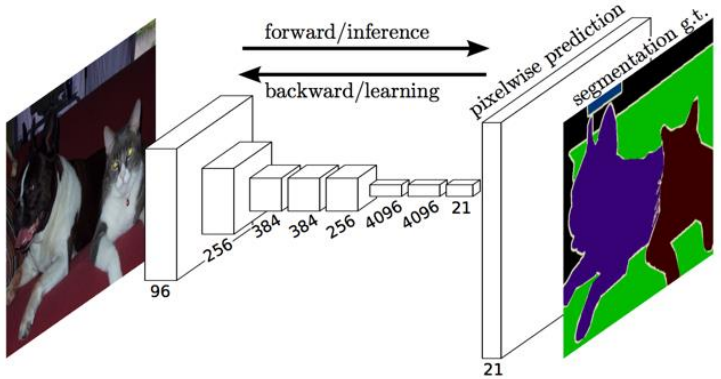
- **Encoder** (downsample): conv + pool → rich **semantic** features, location is lost
- **Decoder** (upsample): recovers the **resolution** to give a full-size map
- Cheaper than full-resolution convs, and a **large receptive field** for context




*Sample Architecture of a FCN with in network downsampling & upsampling (Source – Ref No 3)*

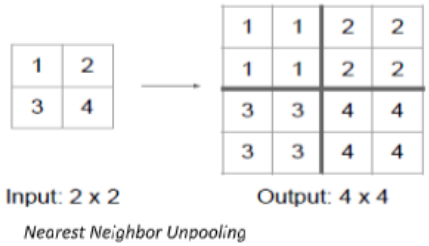
# Fully Convolutional Networks (FCN, 2015)

- FCN = a CNN **without fully connected layers**
- Replace FC layers by **1×1 convolutions** → output is a **spatial map** (a heat map per class), not one vector
  - **Standard FC Layer: e.g.,** [7 x 7 x 512] ---> Flatten ---> Dense Layer ---> Output: [10] (1D Vector)
  - **1x1 Convolution:** **e.g.,** [7 x 7 x 512] ---> 1x1 Conv (10 filters) ---> Output: [7 x 7 x 10] (Spatial Heatmap)
    - +-- Slice 0: [7 x 7] Cat Heatmap
    - +-- Slice 1: [7 x 7] Dog Heatmap
    - -- ...
- Works on **any input size**
- Trained end to end with a **pixel-wise loss** on the output map

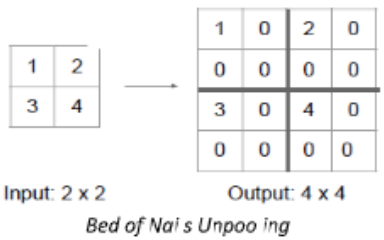



  - The model is trained **end-to-end** using a **pixel-wise loss function** (such as pixel-wise cross-entropy comparing predicted classes to ground-truth labels at every single pixel position).

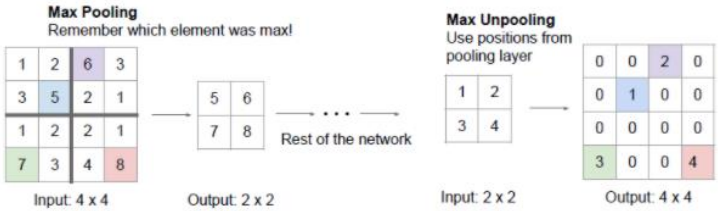
# Upsampling with Fixed Rules




Nearest neighbour: copy values



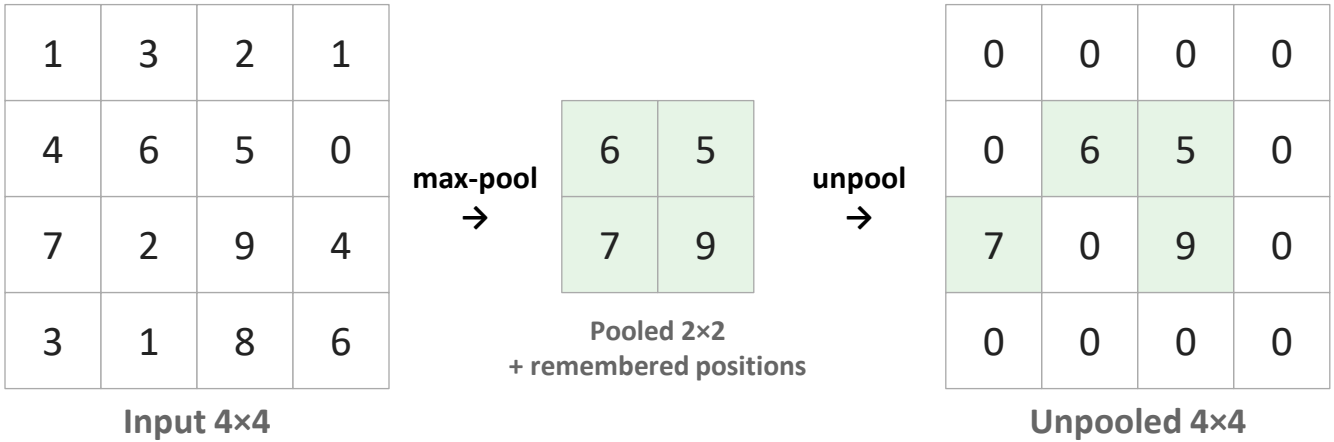

Bed of nails: zeros elsewhere




Max-unpool: use stored max positions

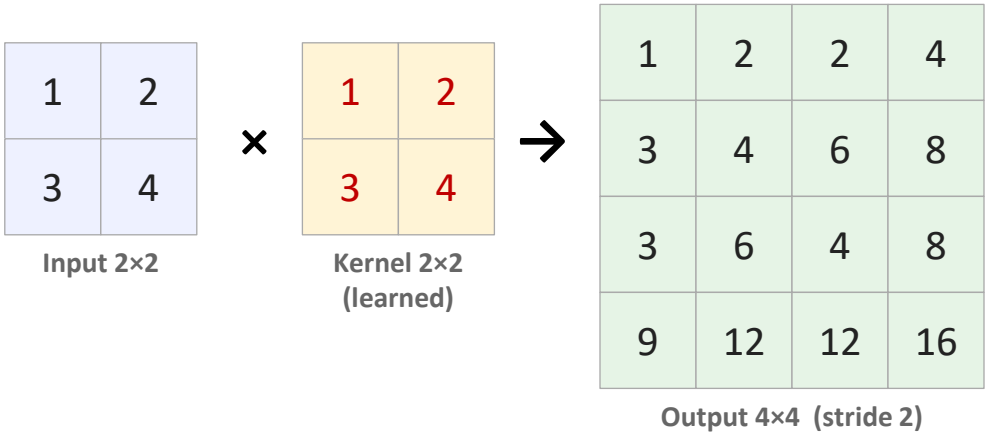
- **Nearest neighbour**: each value is **copied** into a 2×2 block (1 → 1 1 / 1 1). Simple, blocky.
- **Bed of nails**: each value goes to the top-left of its 2×2 block, the rest are **zeros**. Sparse; later convs fill the gaps.
- **Max-unpooling**: put each value back at the **position of the original maximum** (remembered during max-pool); zeros elsewhere. Keeps **where** the strongest response was (used by SegNet).

# Max-Pooling and Max-Unpooling Step by Step



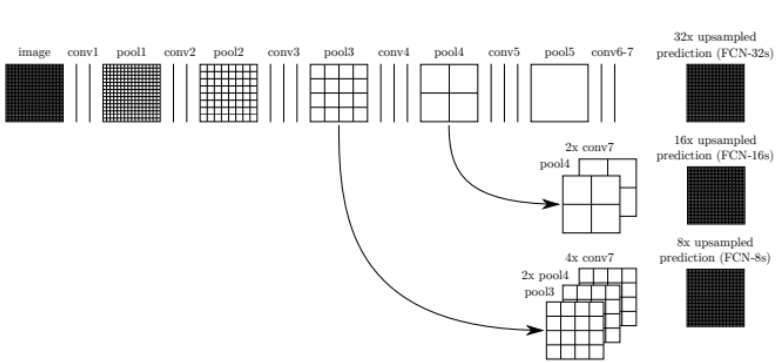

- The pool took the maximum of each 2×2 block and **stored its index** (which of the 4 positions).
- Unpooling writes each value back to its **stored position**, zeros elsewhere.
- SegNet uses this; the stored indices cost only **2 bits per window**.

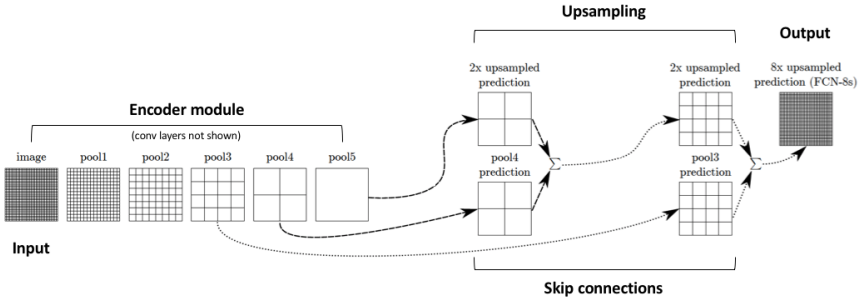
# Transposed Convolution: Learned Upsampling




- Each input value **multiplies the whole kernel** and the result is **stamped** into the output
  - top-left block = 1×kernel,
  - next = 2 × kernel, …).
- Where stamps **overlap** (kernel larger than stride) values are **summed**.
- Kernel weights are **learned**, so the net learns the best way to upsample.
- Often (wrongly) called "deconvolution";
  - it is **not** the inverse of convolution.

# FCN-32s, FCN-16s, FCN-8s: Fusing Skip Information

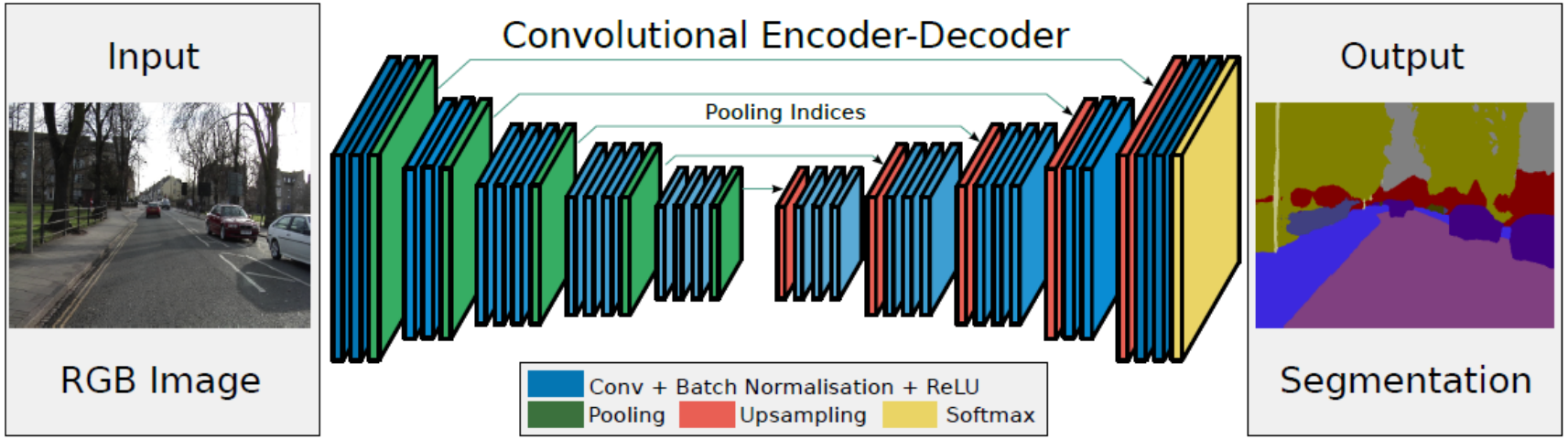



| Variant | How it works |
|---|---|
| FCN-32s | Upsample the last map (stride 32) ×32: coarse (Stride 32 = one output value per 32×32 pixels) |
| FCN-16s | fuse with pool4, upsample ×16 |
| FCN-8s | also fuse pool3, upsample ×8 (Fine-grained, sharp boundaries) |

**Idea: Deep layers say WHAT, shallow layers say WHERE.**

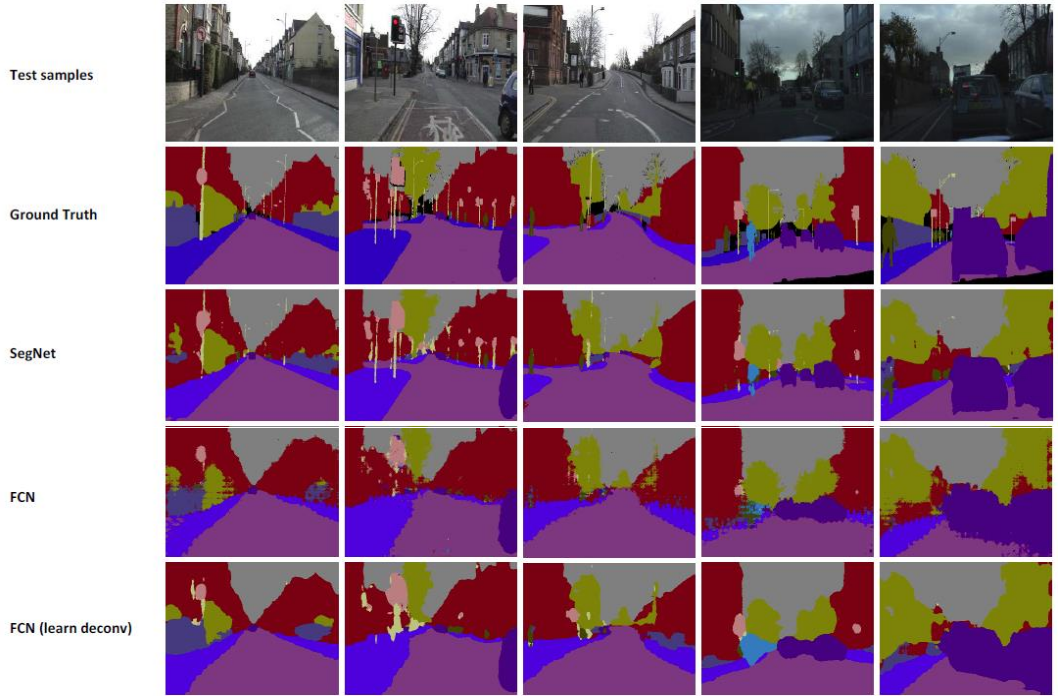
# SegNet (2015)




- Encoder = **13 conv layers of VGG-16** (5 max-pool stages; FC layers dropped); decoder mirrors it
- Encoder **stores max-pooling indices**; decoder uses them to unpool: no learned upsampling, **memory-efficient**
- Final **K-class softmax per pixel**

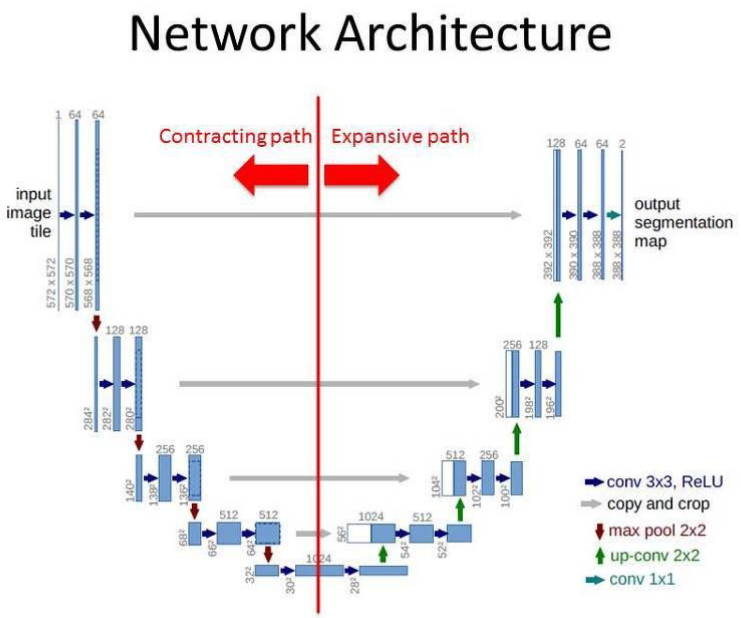
# Limits of SegNet, and the Fix

- Indices carry **position**, not **content**: fine edge detail is only partly recovered.
- **Rare / small classes** are often lost: class imbalance (Lecture 3).
- All information must squeeze through the **bottleneck**, and gradients weaken over the long path.
- **Fix**: pass **encoder features directly** to the decoder through skip connections → **U-Net**.




Example: same road scenes segmented by several methods, differing in how sharp the boundaries are.

# U-Net (2015): Overview




- Made for **biomedical** images with **very few annotated samples**.
- **Contracting path** (left): conv blocks + max-pool; channels **double** (64 → 1024), size halves.
  - a U shape
- **Expansive path** (right): 2×2 up-conv (halves channels, doubles size) + conv blocks.
- **Skip connections** (gray arrows): encoder features are **cropped** and **concatenated** to the decoder at the same level.
- **Head**: 1×1 conv → K class maps.

# U-Net Dimensions (Unpadded Convs)

| Level | Encoder sizes (3×3 conv, no padding) | Channels | Decoder sizes |
|---|---|---|---|
| 1 | 572 → 570 → 568, pool → 284 | 64 | 392 → 390 → 388 (output) |
| 2 | 284 → 282 → 280, pool → 140 | 128 | 200 → 198 → 196 |
| 3 | 140 → 138 → 136, pool → 68 | 256 | 104 → 102 → 100 |
| 4 | 68 → 66 → 64, pool → 32 | 512 | 56 → 54 → 52 |
| Bottom | 32 → 30 → 28 | 1024 | up-conv ×2 → 56 |

- **Each unpadded 3×3 conv loses 2 pixels** (1 per border), so output (388×388) is smaller than input (572×572).
- Skip features are **center-cropped** to the decoder size before concatenation (e.g. 64×64 cropped to 56×56).
- Modern code uses **same padding**, so sizes stay equal and no cropping is needed.

# FCN vs SegNet vs U-Net

| | FCN | SegNet | U-Net |
|---|---|---|---|
| Upsampling | Transposed conv | Max-unpool (stored indices) | Up-conv (transposed conv) |
| Skip links | Add pool3 / pool4 scores | Indices only, no features | Concatenate full feature maps |
| Backbone | VGG-16 | VGG-16 (13 conv) | Symmetric, trained from scratch |
| Strength | First end-to-end deep segmentation | Memory-efficient | Sharp boundaries, small data |
| Weakness | Coarse output (stride 8) | Blurry edges, rare classes lost | Heavier memory (concatenations) |
| Typical use | Baseline | Road scenes | Medical, and many later models |

# Loss Functions for Segmentation

### Pixel-wise cross-entropy

Average cross-entropy over all pixels. Simple, but dominated by **background** pixels.

### Weighted cross-entropy

Larger weight for rare classes or boundary pixels (**class imbalance**, Lecture 3).

### Dice loss

Optimizes **overlap** directly; robust to imbalance.

# Metrics: Pixel Accuracy, IoU and Dice

| | | | |
|---|---|---|---|
| 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

Ground truth

| | | | |
|---|---|---|---|
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

Prediction

| Count | Meaning | Example Here |
|---|---|---|
| TP | predicted 1, truly 1 | 2 |
| FP | predicted 1, truly 0 | 2 |
| FN | predicted 0, truly 1 | 2 |
| TN | predicted 0, truly 0 | 10 |

| Metric | Formula | Value |
|---|---|---|
| Pixel accuracy | (TP+TN)/all | 12/16 = 75% |
| IoU | TP/(TP+FP+FN) | 2/6 = 0.33 |
| Dice | 2TP/(2TP+FP+FN) **= 2·IoU/(1+IoU)** | 4/8 = 0.50 |

# Metrics: Pixel Accuracy, IoU, and Dice

| | | | |
|---|---|---|---|
| 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

Ground truth

| | | | |
|---|---|---|---|
| 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

Prediction

- Intersection = **2**, union = **6**
- **IoU** = TP / (TP+FP+FN) = 2/6 = **0.33**
- **Dice** = 2TP / (2TP+FP+FN) = 4/8 = **0.50**
- **Pixel accuracy** = 12/16 = 75%: misleading

**Use IoU / Dice, not pixel accuracy, when the background dominates.**

# Common Datasets and Benchmarks

| Dataset | Content | Classes | Size / note |
|---|---|---|---|
| **PASCAL VOC** | general objects | 21 (20 + background) | ~ 10k annotated images; classic benchmark |
| **Cityscapes** | urban driving scenes | 19 evaluated (30 defined) | 5,000 finely + 20,000 coarsely annotated, 2048×1024 |
| **ADE20K** | indoor / outdoor scenes | 150 (SceneParsing) | ~ 20k training images |
| **Medical (ISBI etc.)** | cells, organs | 2-10 | tens to hundreds of images |

# Beyond These Models (Self-Read)

## DeepLab

**Atrous (dilated) conv**: spaces the kernel taps by rate r; a 3×3 kernel with r = 2 covers 5×5 with still 9 weights. **ASPP** applies several rates in parallel for multi-scale context.

## PSPNet

**Pyramid pooling**: pool the feature map to 1×1, 2×2, 3×3, 6×6 grids, upsample and concatenate: adds **global scene context**.

## Mask R-CNN

Detection + a **mask branch** per box: **instance** segmentation

## Transformers

**SegFormer, Mask2Former**: attention-based decoders. **SAM**: segments any object from a click or box prompt.

# Check Yourself: Questions and Answers

| Question | Answer |
| --- | --- |
| Why not run full-resolution convs everywhere? | Memory (≈ 67 MB per 512×512×64 map) and a tiny receptive field. |
| How does an FC layer become a conv? | Treat its weights as filters covering the whole input map; it then slides over larger inputs. |
| What do skip connections add in U-Net? | Sharp encoder detail concatenated into the decoder. |
| Pixel accuracy 95% but IoU 0.1. Why? | Background dominates; the object is barely overlapped. |

# Lecture 7 Summary

- Semantic segmentation = **a class for every pixel**
- **Encoder-decoder**: shrink for context and cost, grow back for resolution.
- **FCN**: FC layers become 1×1 convs; **upsampling** by interpolation, unpooling or transposed conv.
- **SegNet** reuses pooling indices; **U-Net** concatenates encoder features, so boundaries stay sharp.
- Train say with **cross-entropy or Dice**; evaluate with **IoU, Dice,** not pixel accuracy.
- Next: **object detection**